

```
1 #include <windows.h>
2 #include <stdio.h>
3
4 #include <gl\glew.h>
5
6 #include <gl\GL.h>
7
8 #pragma comment(lib, "glew32.lib")
9 #pragma comment(lib, "opengl32.lib")
10
11 #define WIN_WIDTH 800
12 #define WIN_HEIGHT 600
13
14 //Prototype Of WndProc() declared Globally
15 LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
16
17 //Global variable declarations
18 FILE *gpFile = NULL;
19
20 HWND ghwnd = NULL;
21 HDC ghdc = NULL;
22 HGLRC ghrc = NULL;
23
24 DWORD dwStyle;
25 WINDOWPLACEMENT wpPrev = { sizeof(WINDOWPLACEMENT) };
26
27 bool gbActiveWindow = false;
28 bool gbEscapeKeyIsPressed = false;
29 bool gbFullscreen = false;
30
31 //main()
32 int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdLine, int iCmdShow)
33 {
34     //function prototype
35     void initialize(void);
36     void uninitialize(void);
37     void display(void);
38
39     //variable declaration
40     WNDCLASSEX wndclass;
41     HWND hwnd;
42     MSG msg;
43     TCHAR szClassName[] = TEXT("OpenGLPP");
44     bool bDone = false;
45
46     //code
47     if (fopen_s(&gpFile, "Log.txt", "w") != 0)
48     {
49         MessageBox(NULL, TEXT("Log File Can Not Be Created\nExiting ..."), TEXT("Error"), MB_OK | MB_TOPMOST | MB_ICONSTOP);
50         exit(0);
51     }
52     else
53     {
54         fprintf(gpFile, "Log File Is Successfully Opened.\n");
```

```
55     }
56
57     wndclass.cbSize = sizeof(WNDCLASSEX);
58     wndclass.style = CS_HREDRAW | CS_VREDRAW | CS_OWNDC;
59     wndclass.cbClsExtra = 0;
60     wndclass.cbWndExtra = 0;
61     wndclass.hInstance = hInstance;
62     wndclass.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
63     wndclass.hIcon = LoadIcon(NULL, IDI_APPLICATION);
64     wndclass.hCursor = LoadCursor(NULL, IDC_ARROW);
65     wndclass.hIconSm = LoadIcon(NULL, IDI_APPLICATION);
66     wndclass.lpfnWndProc = WndProc;
67     wndclass.lpszClassName = szClassName;
68     wndclass.lpszMenuName = NULL;
69
70     RegisterClassEx(&wndclass);
71
72     hwnd = CreateWindowEx(WS_EX_APPWINDOW,
73         szClassName,
74         TEXT("OpenGL Programmable Pipeline : Window"),
75         WS_OVERLAPPEDWINDOW | WS_CLIPCHILDREN | WS_CLIPSIBLINGS | WS_VISIBLE,
76         100,
77         100,
78         WIN_WIDTH,
79         WIN_HEIGHT,
80         NULL,
81         NULL,
82         hInstance,
83         NULL);
84
85     ghwnd = hwnd;
86
87     ShowWindow(hwnd, iCmdShow);
88     SetForegroundWindow(hwnd);
89     SetFocus(hwnd);
90
91     initialize();
92
93     while (bDone == false)
94     {
95         if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
96         {
97             if (msg.message == WM_QUIT)
98                 bDone = true;
99             else
100             {
101                 TranslateMessage(&msg);
102                 DispatchMessage(&msg);
103             }
104         }
105         else
106         {
107             display();
108
109             if (gbActiveWindow == true)
110             {
```

```
111         if (gbEscapeKeyIsPressed == true)
112             bDone = true;
113     }
114 }
115 }
116
117     uninitialized();
118
119     return((int)msg.wParam);
120 }
121
122 //WndProc()
123 LRESULT CALLBACK WndProc(HWND hwnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
124 {
125     //function prototype
126     void resize(int, int);
127     void ToggleFullscreen(void);
128     void uninitialized(void);
129
130     //code
131     switch (iMsg)
132     {
133     case WM_ACTIVATE:
134         if (HIWORD(wParam) == 0)
135             gbActiveWindow = true;
136         else
137             gbActiveWindow = false;
138         break;
139     case WM_ERASEBKGD:
140         return(0);
141     case WM_SIZE:
142         resize(LOWORD(lParam), HIWORD(lParam));
143         break;
144     case WM_KEYDOWN:
145         switch (wParam)
146         {
147         case VK_ESCAPE:
148             gbEscapeKeyIsPressed = true;
149             break;
150         case 0x46:
151             if (gbFullscreen == false)
152             {
153                 ToggleFullscreen();
154                 gbFullscreen = true;
155             }
156             else
157             {
158                 ToggleFullscreen();
159                 gbFullscreen = false;
160             }
161             break;
162         default:
163             break;
164         }
165         break;
166     case WM_LBUTTONDOWN:
```

```
167         break;
168     case WM_CLOSE:
169         uninitialize();
170         break;
171     case WM_DESTROY:
172         PostQuitMessage(0);
173         break;
174     default:
175         break;
176     }
177     return(DefWindowProc(hwnd, iMsg, wParam, lParam));
178 }
179
180 void ToggleFullscreen(void)
181 {
182     //variable declarations
183     MONITORINFO mi;
184
185     //code
186     if (gbFullscreen == false)
187     {
188         dwStyle = GetWindowLong(ghwnd, GWL_STYLE);
189         if (dwStyle & WS_OVERLAPPEDWINDOW)
190         {
191             mi = { sizeof(MONITORINFO) };
192             if (GetWindowPlacement(ghwnd, &wpPrev) && GetMonitorInfo
193                 (MonitorFromWindow(ghwnd, MONITORINFOF_PRIMARY), &mi))
194             {
195                 SetWindowLong(ghwnd, GWL_STYLE, dwStyle &
196                     ~WS_OVERLAPPEDWINDOW);
197                 SetWindowPos(ghwnd, HWND_TOP, mi.rcMonitor.left,
198                     mi.rcMonitor.top, mi.rcMonitor.right - mi.rcMonitor.left,
199                     mi.rcMonitor.bottom - mi.rcMonitor.top, SWP_NOZORDER |
200                     SWP_FRAMECHANGED);
201             }
202             ShowCursor(FALSE);
203         }
204     }
205     else
206     {
207         //code
208         SetWindowLong(ghwnd, GWL_STYLE, dwStyle | WS_OVERLAPPEDWINDOW);
209         SetWindowPlacement(ghwnd, &wpPrev);
210         SetWindowPos(ghwnd, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE | SWP_NOSIZE |
211             SWP_NOOWNERZORDER | SWP_NOZORDER | SWP_FRAMECHANGED);
212         ShowCursor(TRUE);
213     }
214 }
215
216 void initialize(void)
217 {
218     //function prototypes
219     void uninitialize(void);
220     void resize(int, int);
```

```
217
218     //variable declarations
219     PIXELFORMATDESCRIPTOR pfd;
220     int iPixelFormatIndex;
221
222     //code
223     ZeroMemory(&pfd, sizeof(PIXELFORMATDESCRIPTOR));
224
225     pfd.nSize = sizeof(PIXELFORMATDESCRIPTOR);
226     pfd.nVersion = 1;
227     pfd.dwFlags = PFD_DRAW_TO_WINDOW | PFD_SUPPORT_OPENGL | PFD_DOUBLEBUFFER;
228     pfd.iPixelFormat = PFD_TYPE_RGBA;
229     pfd.cColorBits = 32;
230     pfd.cRedBits = 8;
231     pfd.cGreenBits = 8;
232     pfd.cBlueBits = 8;
233     pfd.cAlphaBits = 8;
234     pfd.cDepthBits = 32;
235
236     ghdc = GetDC(ghwnd);
237
238     iPixelFormatIndex = ChoosePixelFormat(ghdc, &pfd);
239     if (iPixelFormatIndex == 0)
240     {
241         ReleaseDC(ghwnd, ghdc);
242         ghdc = NULL;
243     }
244
245     if (SetPixelFormat(ghdc, iPixelFormatIndex, &pfd) == false)
246     {
247         ReleaseDC(ghwnd, ghdc);
248         ghdc = NULL;
249     }
250
251     ghrc = wglCreateContext(ghdc);
252     if (ghrc == NULL)
253     {
254         ReleaseDC(ghwnd, ghdc);
255         ghdc = NULL;
256     }
257
258     if (wglMakeCurrent(ghdc, ghrc) == false)
259     {
260         wglDeleteContext(ghrc);
261         ghrc = NULL;
262         ReleaseDC(ghwnd, ghdc);
263         ghdc = NULL;
264     }
265
266     GLenum glew_error = glewInit();
267     if (glew_error != GLEW_OK)
268     {
269         wglDeleteContext(ghrc);
270         ghrc = NULL;
271         ReleaseDC(ghwnd, ghdc);
272         ghdc = NULL;
```

```
273     }
274
275     glShadeModel(GL_SMOOTH);
276     glClearDepth(1.0f);
277     glEnable(GL_DEPTH_TEST);
278     glDepthFunc(GL_LEQUAL);
279     glHint(GL_PERSPECTIVE_CORRECTION_HINT, GL_NICEST);
280     glEnable(GL_CULL_FACE);
281
282     glClearColor(0.0f, 0.0f, 1.0f, 0.0f);
283
284     resize(WIN_WIDTH, WIN_HEIGHT);
285 }
286
287 void display(void)
288 {
289     //code
290     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
291
292     // OpenGL Drawing
293
294     SwapBuffers(ghdc);
295 }
296
297 void resize(int width, int height)
298 {
299     //code
300     if (height == 0)
301         height = 1;
302     glViewport(0, 0, (GLsizei)width, (GLsizei)height);
303 }
304
305 void uninitialize(void)
306 {
307     //code
308     if (gbFullscreen == true)
309     {
310         dwStyle = GetWindowLong(ghwnd, GWL_STYLE);
311         SetWindowLong(ghwnd, GWL_STYLE, dwStyle | WS_OVERLAPPEDWINDOW);
312         SetWindowPlacement(ghwnd, &wpPrev);
313         SetWindowPos(ghwnd, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE | SWP_NOSIZE |
314                     SWP_NOOWNERZORDER | SWP_NOZORDER | SWP_FRAMECHANGED);
315
316         ShowCursor(TRUE);
317     }
318
319     wglMakeCurrent(NULL, NULL);
320
321     wglDeleteContext(ghrc);
322     ghrc = NULL;
323
324     ReleaseDC(ghwnd, ghdc);
325     ghdc = NULL;
326
327     if (gpFile)
```

```
328     {  
329         fprintf(gpFile, "Log File Is Successfully Closed.\n");  
330         fclose(gpFile);  
331         gpFile = NULL;  
332     }  
333 }  
334
```